Optimization Autopsy

An error-aware counterfactual notebook for compiler optimization

Core idea: make each optimization inspectable as an experiment. Show what changed, whether the observed behavior survived, and what happens when that pass is removed.

What is the proposed novelty?

OptiTAC combines pass-by-pass replay, runtime-error preservation, leave-one-pass-out experiments and a portable evidence record in a source-to-TAC learning workbench. The proposed contribution is this integrated investigation workflow and its potential educational value.

A conventional before/after display shows the final transformation. OptiTAC also lets a user examine intermediate checkpoints, locate the earliest observed divergence, and compare a pipeline against versions with individual passes omitted.

The five optimization algorithms themselves are established techniques. This document does not claim that pass debugging, translation validation, omission experiments or fault injection were invented by OptiTAC. Global uniqueness and publication novelty require a wider comparison and empirical evaluation.

Five parts of one investigation

1. Transformation replay. After every selected pass, the engine stores independent before/after TAC snapshots, instruction IDs, rewrite explanations and the pass round. Even a pass with no rewrite remains visible.

2. Error-aware observation. The original and transformed TAC are executed with the same interpreter. Successful return values are compared with Object.is; failed executions are compared by their error messages. A preserved error is labelled as an error, not as successful execution.

3. First-divergence localization. Each checkpoint is compared with the original observation. The earliest unequal checkpoint is retained, even if a later transformation masks the difference. This identifies a recorded divergence, not a minimal root cause or formal causal proof.

4. Counterfactual omission. The entire selected pipeline is rerun once per enabled pass, with that pass removed. The workbench reports final TAC count, count difference and preservation of the original observation.

5. Exportable evidence. JSON contains source, selections, tokens, AST, TAC, checkpoints, logs and execution observations. A labelled return-value fault can be injected after optimization to demonstrate detection. It is excluded from omission comparisons.

> The distinctive experience is the connection: inspect a rewrite, inspect its semantic observation, then challenge its contribution by removing the pass.

Implementation: src/compiler/lab.ts orchestrates the experiment; src/App.tsx exposes Autopsy, Counterfactual and Structure views. Selected passes run in a fixed order until unchanged or ten rounds. A run records at most 50 ordinary checkpoints plus one optional injected-fault checkpoint.

Optimization is more than shorter code

Example A - A propagation cascade

```
int a = 10;
int b = a + 5;
int spare = 99;
int c = b * 2;
return c;
```

With all passes enabled, this example reduces from seven TAC instructions to return 30. Replay shows the intermediate substitutions and folds. Omission experiments reveal what remains when a pass is unavailable; another pass may compensate across rounds.

Example B - The unused computation that matters

```
int a = 8;
int unused = a / 0;
return a;
```

The original execution fails with division by zero. Deleting the unused division would incorrectly change failure into return 8 under this interpreter. OptiTAC retains potentially trapping division and modulo operations, so the optimized program preserves the error.

> This example exposes the central teaching point: a value can be unused while its computation still has observable behavior.

Example C - A controlled wrong result

For the cascade, the optional fault changes the return from 30 to 31 after normal optimization. Autopsy identifies the injected checkpoint as the first divergence. This is a deliberately seeded demonstration, not evidence of discovering a new compiler bug. On an erroring program, execution may never reach the modified return.

These observations apply to the project interpreter. In particular, division-by-zero behavior here must not be generalized to C undefined behavior.

What is established, and what is proposed?

Approach	Established capability	OptiTAC positioning
LLVM OptBisect [1]	Selectively disables optimizations to help locate errors.	Uses visible replay and full-pipeline omission experiments in a small teaching notebook.
LLVM remarks [2]	Reports information about optimization transformations.	Places rewrite explanations beside concrete outcomes and counterfactual instruction counts.
Alive2 [3,4]	Provides bounded translation validation for LLVM.	Checks concrete closed-program observations; does not match the strength of symbolic validation.

A defensible novelty statement

> OptiTAC investigates an integrated error-aware counterfactual notebook for compiler education, linking pass-local rewrites, observed behavioral preservation and omission-based contribution analysis in a single inspectable workflow.

This is a candidate system and educational contribution. The comparison above establishes related foundations; it does not establish that no other tool combines them. A systematic survey of educational compilers and optimization-debugging interfaces is still needed.

Instruction-count differences are not runtime speedups. Omission effects include interactions and are not additive. A zero difference can mean redundancy, compensation by another pass, or lack of applicability.

What the implementation supports

Build and lint checks pass. Thirteen automated test cases cover the experiment pipeline, including a deterministic corpus of forty arithmetic programs, zero-division and zero-modulo traps, injected divergence, omission effects, snapshot isolation and numeric handling. Browser checks cover desktop/mobile behavior, error messages and JSON export. These are engineering checks, not a comparative research study.

The language supports initialized int/float declarations, arithmetic, parentheses and returns. Arithmetic uses JavaScript Number, not fixed-width C integer semantics. There are no external runtime inputs; branching, loops, CFG analysis, symbolic verification, reduced counterexamples and persistent experiment history are not implemented.

What would make a research claim convincing?

Compare three interfaces: final before/after only; replay with outcomes; and the full replay-plus-omission workflow. Use a reproducible set of arithmetic programs, seeded transformation faults, numeric edge cases and observable-error cases. Measure fault detection, checkpoint-localization accuracy, explanation accuracy and time to complete investigation tasks.

An appropriately designed learner study could test whether omission experiments improve understanding of pass interactions. Report participant selection, consent, task difficulty, uncertainty and failure cases. No participant data, measured learning benefit or publication acceptance currently exists.

References

[1] LLVM. Using -opt-bisect-limit to debug optimization errors. https://llvm.org/docs/OptBisect.html

[2] LLVM. Remarks. https://llvm.org/docs/Remarks.html

[3] AliveToolkit. Alive2 project documentation. https://github.com/AliveToolkit/alive2

[4] Lopes et al. Alive2: Bounded Translation Validation for LLVM. PLDI 2021. https://users.cs.utah.edu/~regehr/alive2-pldi21.pdf

Sources checked 13 October 2026. Project claims describe the inspected implementation.